# Gastronomie Créative - bilan de développement

Récapitulatif des 27 et 28 octobre 2026
Projet : ASP.NET Core MVC (.NET 10), C#, JavaScript, Entity Framework Core, PostgreSQL et DBeaver.

État général : la transmission des ingrédients au contrôleur est validée et la recette de test est enregistrée. L'enregistrement des associations dans ingredients_recettes reste à diagnostiquer.

## 1. Gestion dynamique des ingrédients en JavaScript

Le formulaire de création de recette permet de rechercher les ingrédients disponibles, de les filtrer avec filter() et includes(), d'éviter les doublons avec some(), puis de créer dynamiquement les champs HTML de sélection et de quantité. La suppression enlève à la fois les éléments visibles et l'entrée du tableau IngredientsSelectionnes.

Pour que le Model Binding ASP.NET Core reçoive une liste cohérente, les noms des champs doivent être indexés :

```
IngredientsSelectionnes[0].IdIngredient
IngredientsSelectionnes[0].Quantite
IngredientsSelectionnes[1].IdIngredient
IngredientsSelectionnes[1].Quantite
```

Après une suppression, la fonction MettreAJourIndices() renumérote les champs. Chaque objet conserve les références vers ses éléments HTML :

```
ingredient.champId = IdIngredientHTML;
ingredient.champQuantite = QuantiteIngredientHTML;
```

```
function MettreAJourIndices() {
  IngredientsSelectionnes.forEach(function(ingredient, index) {
    ingredient.champId.name =
      `IngredientsSelectionnes[${index}].IdIngredient`;
    ingredient.champQuantite.name =
      `IngredientsSelectionnes[${index}].Quantite`;
  });
}
```

## 2. Model Binding et vérification par le débogueur

Le ViewModel RecetteCreationViewModel contient une liste de IngredientRecetteViewModel, dont chaque élément comporte IdIngredient et Quantite. Les champs HTML correctement nommés sont automatiquement convertis en objets C# lors du POST.

```
public List<IngredientRecetteViewModel> IngredientsSelectionnes
    { get; set; } = new();

public int IdIngredient { get; set; }
public string Quantite { get; set; } = string.Empty;
```

Dans VS Code, lancement de la configuration C# GC_MVC, validation du certificat HTTPS de développement local, utilisation de F5 et d'un point d'arrêt à if (ModelState.IsValid). Inspection des variables via Run and Debug > Variables > Locals.

| Index | IdIngredient | Quantite |
|---|---|---|
| 0 | 284 | 300 g |
| 1 | 209 | 2 g |

| Index | IdIngredient | Quantite |
|---|---|---|
| 2 | 18 | une cuillère à soupe |

Résultat confirmé : IngredientsSelectionnes.Count = 3, avec les trois identifiants et les trois quantités attendus. Ce test prouve que les données sont bien parvenues au contrôleur à cette occasion.

## 3. Vérification préalable de l’existence des ingrédients

Avant la transaction, une boucle foreach parcourt les objets envoyés par le formulaire. Any() vérifie qu’un ingrédient correspondant existe dans PostgreSQL. Le symbole ! inverse le booléen : if (!ingredientExiste) traite le cas où l’ingrédient n’existe pas.

```
foreach (IngredientRecetteViewModel ingredient
        in model.IngredientsSelectionnes) {
  bool ingredientExiste = _context.Ingredients.Any(
    ingredientBDD =>
      ingredientBDD.id_ingredient == ingredient.IdIngredient);

  if (!ingredientExiste) {
    ModelState.AddModelError("IngredientsSelectionnes",
      "Un ingrédient sélectionné n’existe plus.");
    model.IngredientsDisponibles = _context.Ingredients.ToList();
    return View("CreerRecette", model);
  }
}
```

Notions étudiées : différence entre Ingredients (entité BDD), IngredientRecetteViewModel (données du formulaire) et Ingredients_recettes (association recette-ingrédient) ; booléens true/false ; conditions if ; retour de vue et arrêt de l’action. Une erreur d’accolades provoquant un retour prématuré a été corrigée.

## 4. Création des associations et transaction EF Core

Le contrôleur enregistre d’abord la nouvelle recette et appelle SaveChanges() pour récupérer son identifiant. Il prépare ensuite les associations de catégories et les associations d’ingrédients, effectue un second SaveChanges(), puis valide la transaction avec Commit().

```
foreach (IngredientRecetteViewModel ingredient
        in model.IngredientsSelectionnes) {
  Ingredients_recettes NouvelleAssociationIngredient =
    new Ingredients_recettes {
      id_ingredient = ingredient.IdIngredient,
      id_recette = NouvelleRecette.id_recette,
      quantite = ingredient.Quantite
    };
  _context.Ingredients_Recettes.Add(NouvelleAssociationIngredient);
}
_context.SaveChanges();
```

La différence entre Add() (mise en attente de l’insertion) et SaveChanges() (envoi à la base) a été expliquée. Le code d’association est bien placé avant Commit() dans le dernier fichier examiné, RecettesController(6).cs. La commande dotnet build a réussi.

## 5. Tests réels dans DBeaver

Après la soumission du formulaire, le navigateur est revenu à la page d’accueil sans afficher de message de succès. La présence de TempData["MessageSucces"] dans le contrôleur ne garantit pas son affichage : la vue cible doit le rendre visible.

La recherche SQL a confirmé l’existence de la recette de test :

```
SELECT id_recette, nom
FROM recettes
WHERE nom = 'test enregistrement ingredient';
```

| id_recette | nom |
|---|---|
| 63 | test enregistrement ingredient |

La recherche des associations pour cette recette a renvoyé zéro ligne :

```
SELECT id_recette, id_ingredient, quantite
FROM ingredients_recettes
WHERE id_recette = 63;
```

Conclusion vérifiée : la recette n° 63 est présente dans recettes, mais aucune association correspondante n'a été retrouvée dans ingredients_recettes lors de ce test. La cause exacte n'est pas encore établie.

## 6. Examen du contrôleur et points à améliorer

Le fichier RecettesController(6).cs comporte bien la vérification préalable des catégories et ingrédients, la création de la recette, les deux boucles d'associations, les deux appels à SaveChanges() et Commit(). Aucune erreur évidente de placement de la boucle des ingrédients n'a été observée.

Des retours vers le formulaire rechargent seulement CategoriesDisponibles ou seulement IngredientsDisponibles. Il faudra harmoniser ces retours pour recharger les deux listes en cas d'erreur. Ce point ne démontre toutefois pas la cause des associations manquantes.

## 7. Reprise prévue le 29 octobre 2026

Point d'arrêt prévu : ligne 238, sur le foreach de création des associations d'ingrédients. Procédure : démarrer F5, créer une nouvelle recette avec deux ingrédients et leurs quantités, vérifier que le point d'arrêt est atteint, inspecter model.IngredientsSelectionnes.Count, suivre la boucle et le second SaveChanges(), puis rechercher les associations du nouvel identifiant dans DBeaver.

Hypothèses à tester, sans conclure prématurément : la liste pourrait être vide lors du dernier POST, ou le test pourrait avoir utilisé une version de l'application différente du code inspecté.

## 8. Compétences pratiquées

| Domaine | Notions |
|---|---|
| JavaScript | Tableaux, objets, filter, includes, some, forEach, DOM, événements |
| HTML / MVC | Champs dynamiques, attribut name, champs cachés, Model Binding |
| C# | Objets, propriétés, foreach, booléens, if, ViewModels |
| EF Core | Any, Add, SaveChanges, transaction, Commit, Rollback |
| PostgreSQL / DBeaver | Tables de liaison, SELECT, vérification de données |
| VS Code | dotnet build, F5, points d'arrêt, Locals |

Priorité de demain : identifier pourquoi la boucle des ingrédients n'a pas produit de lignes dans ingredients_recettes, en conservant tout le fonctionnement déjà validé.